Reporte de Estadía

Para obtener el título de

Licenciatura en Ingeniería en Entornos Virtuales y Negocios Digitales.

Proyecto:

Chatbot de Apoyo Académico.

Realizado en la empresa:

Centro de Cómputo Académico de la Universidad Autónoma del Estado de Hidalgo.

Presentado por:

Uriel Ivan Perez Trujillo

Asesor Industrial	**Asesor Académico**
_____	_____
David Muñoz Roa	**Edith Campos Cruz**
Encargado del área de Innovación y Desarrollo	**Profesor de Tiempo Completo**

Mineral de la Reforma, Hidalgo a 17 de abril de 2026.

Contenido

Abstract.....4
Introducción.....5
Descripción de la Empresa.....6
Contexto y Planteamiento del Problema.....11
Objetivo General.....13
Hipótesis.....14
Justificación.....15
Marco teórico.....16
Metodología.....22
Desarrollo del Proyecto.....27
Desarrollo del backend (procesamiento del chatbot).....41
Decisiones importantes durante el desarrollo.....43
Diseños y maquetación de interfaz.....43
Problemas y errores durante el desarrollo.....45
Evaluación del proyecto.....48
Resultados obtenidos en el proyecto.....51
Análisis y discusión de los resultados.....55
Recomendaciones y sugerencias.....57
Conclusiones.....60
Referencias.....62
Anexos.....63

Resumen

El presente proyecto se desarrolla en el área de Innovación y Desarrollo del Centro de Cómputo Académico de la Universidad Autónoma del Estado de Hidalgo, con el propósito de diseñar e implementar una herramienta tecnológica que apoye el aprendizaje del álgebra en estudiantes de nivel preparatoria.

Actualmente, uno de los principales problemas en el ámbito educativo es la dificultad que presentan los estudiantes para comprender temas matemáticos, especialmente aquellos relacionados con el álgebra. Esta situación se agrava cuando los alumnos no cuentan con apoyo inmediato fuera del aula para resolver sus dudas.

Para dar solución a esta problemática, se propone el desarrollo de GarzaMath, un chatbot académico basado en inteligencia artificial que permite a los usuarios realizar preguntas en lenguaje natural y recibir respuestas explicativas en tiempo real.

El sistema se desarrolla mediante el uso de tecnologías web modernas como Next.js y React para la interfaz de usuario, así como Node.js y TypeScript para el backend. Además, se integra un modelo de inteligencia artificial mediante Ollama y se implementa una base de datos utilizando Prisma con SQLite para almacenar las conversaciones.

Como resultado, se obtiene una aplicación funcional que permite a los estudiantes interactuar con un asistente virtual capaz de explicar procedimientos algebraicos, facilitando así su proceso de aprendizaje.

Finalmente, se concluye que el uso de inteligencia artificial aplicada a la educación representa una alternativa viable para fortalecer el aprendizaje autónomo de los estudiantes.

Palabras clave: *inteligencia artificial, chatbot educativo, álgebra, aprendizaje, GarzaMath*

Abstract

This project is developed in the Innovation and Development area of the Academic Computing Center at the Autonomous University of the State of Hidalgo, with the purpose of designing and implementing a technological tool to support the learning of algebra in high school students.

Currently, one of the main problems in education is the difficulty students face in understanding mathematical topics, especially those related to algebra. This situation becomes more complex when students do not have immediate support outside the classroom to solve their doubts.

To address this problem, the development of GarzaMath is proposed, an academic chatbot based on artificial intelligence that allows users to ask questions in natural language and receive explanatory responses in real time.

The system is developed using modern web technologies such as Next.js and React for the user interface, as well as Node.js and TypeScript for the backend. Additionally, an artificial intelligence model is integrated using Ollama, and a database is implemented with Prisma and SQLite to store conversations.

As a result, a functional application is obtained that allows students to interact with a virtual assistant capable of explaining algebraic procedures, thus facilitating their learning process.

Finally, it is concluded that the use of artificial intelligence in education represents a viable alternative to strengthen students' autonomous learning.

Keywords: *artificial intelligence, educational chatbot, algebra, learning, GarzaMath*

Introducción

Las matemáticas forman parte fundamental del proceso educativo, ya que contribuyen al desarrollo del pensamiento lógico, analítico y crítico en los estudiantes. Sin embargo, a pesar de su importancia, esta área del conocimiento continúa representando una de las principales dificultades para los alumnos, especialmente en temas relacionados con el álgebra.

El álgebra, en particular, implica el uso de símbolos, variables y expresiones que requieren un nivel de abstracción mayor en comparación con otras áreas de las matemáticas. Esta característica hace que muchos estudiantes de nivel secundaria y preparatoria presenten problemas al momento de comprender los conceptos y aplicar correctamente los procedimientos necesarios para resolver ejercicios.

En el entorno educativo actual, es común que los estudiantes enfrenten dificultades cuando intentan estudiar de manera independiente. En muchas ocasiones, al realizar tareas o prepararse para exámenes, surgen dudas que no pueden resolver de inmediato, lo que provoca que se generen vacíos en su aprendizaje. Aunque existen diversos recursos disponibles en internet, como videos, tutoriales o páginas educativas, estos no siempre ofrecen una respuesta específica a las necesidades de cada alumno.

Ante esta situación, el uso de tecnologías digitales ha cobrado gran relevancia como apoyo en el proceso de enseñanza-aprendizaje. En particular, la inteligencia artificial ha permitido el desarrollo de herramientas innovadoras que pueden adaptarse a las necesidades del usuario y ofrecer respuestas en tiempo real. Entre estas herramientas destacan los chatbots educativos, los cuales permiten establecer una interacción directa con los estudiantes mediante lenguaje natural.

En este contexto, surge GarzaMath, un chatbot académico diseñado para apoyar a estudiantes de nivel secundaria y preparatoria en la comprensión del álgebra. Este sistema permite a los usuarios realizar preguntas a través de una interfaz de chat y recibir explicaciones automáticas que les ayuden a entender los procedimientos matemáticos de manera más clara.

El desarrollo de este proyecto se llevó a cabo en el Centro de Cómputo Académico de la Universidad Autónoma del Estado de Hidalgo, dentro del área de Innovación y Desarrollo, con el propósito de aplicar tecnologías emergentes en el ámbito educativo y contribuir al fortalecimiento del aprendizaje.

A lo largo del presente reporte se describen las diferentes etapas del desarrollo del sistema, incluyendo el análisis del problema, los objetivos planteados, la metodología utilizada, así como la implementación del chatbot y los resultados obtenidos.

Descripción de la Empresa

1. Antecedentes

La Universidad Autónoma del Estado de Hidalgo (UAEH) es una institución pública de educación superior reconocida a nivel nacional por su compromiso con la formación académica, la investigación y el desarrollo tecnológico. Su origen se remonta al siglo XIX, evolucionando constantemente hasta consolidarse como una de las universidades más importantes del país.

Dentro de su estructura institucional, la UAEH cuenta con diversas áreas enfocadas al soporte académico y tecnológico, entre las cuales destaca el Centro de Cómputo Académico (CECA). Este centro tiene como propósito principal brindar servicios tecnológicos que fortalezcan las actividades educativas y administrativas de la institución.

El CECA surge como respuesta a la necesidad de integrar tecnologías de la información en los procesos educativos, adaptándose a los avances tecnológicos y a las nuevas demandas del entorno académico. A lo largo del tiempo, ha evolucionado desde ofrecer servicios básicos de soporte técnico hasta convertirse en un área estratégica dedicada al desarrollo de sistemas, plataformas digitales y soluciones innovadoras.

Actualmente, el CECA desempeña un papel fundamental en la transformación digital de la universidad, participando en proyectos que impulsan el uso de herramientas tecnológicas en el proceso de enseñanza-aprendizaje.

2. Giro, origen y evolución

El giro principal de la Universidad Autónoma del Estado de Hidalgo es la prestación de servicios educativos en los niveles medio superior y superior, así como el desarrollo de investigación científica y tecnológica.

Dentro de este contexto, el Centro de Cómputo Académico se enfoca en el área tecnológica, proporcionando servicios relacionados con el desarrollo de software, soporte técnico, mantenimiento de infraestructura y creación de soluciones digitales.

En sus inicios, el CECA estaba orientado principalmente al mantenimiento de equipos de cómputo y soporte a usuarios; sin embargo, con el avance de las tecnologías de la información, ha evolucionado hacia un enfoque más integral, incorporando el desarrollo de sistemas web, gestión de bases de datos, implementación de plataformas educativas y uso de tecnologías emergentes.

En la actualidad, el CECA se ha consolidado como un espacio de innovación tecnológica, donde se desarrollan proyectos que contribuyen al fortalecimiento de los procesos educativos, permitiendo además la participación de estudiantes en proyectos reales que fortalecen su formación profesional.

3.Procesos, productos y servicios

El Centro de Cómputo Académico (CECA) ofrece diversos procesos, productos y servicios orientados al soporte tecnológico y al desarrollo de soluciones digitales dentro de la universidad.

3.1 Procesos principales

Entre los procesos más relevantes se encuentran:

- Desarrollo de sistemas web y aplicaciones educativas.
- Mantenimiento y actualización de plataformas institucionales.
- Soporte técnico a usuarios (docentes, administrativos y estudiantes).
- Administración y gestión de bases de datos.
- Implementación de nuevas tecnologías en el entorno educativo.

3.2 Productos

Los principales productos generados por el CECA son:

- Sistemas web institucionales.
- Plataformas digitales de apoyo al aprendizaje.
- Aplicaciones internas para la gestión académica.
- Herramientas tecnológicas innovadoras para la educación.

3.3 Servicios

Entre los servicios que ofrece el CECA destacan:

- Soporte técnico especializado.
- Desarrollo de soluciones tecnológicas a medida.
- Asesoría en implementación de sistemas.
- Capacitación en el uso de herramientas digitales.

4. Procesos del área donde se desarrolla el proyecto

El proceso principal del área de Innovación y Tecnología Educativa inicia con la identificación de una necesidad o problemática dentro del entorno académico. Posteriormente, se realiza un análisis de requerimientos para definir las características del sistema a desarrollar.

Una vez definidos los requerimientos, se procede al diseño de la solución tecnológica, donde se establece la arquitectura del sistema, las herramientas a utilizar y la estructura general del proyecto. En este caso, se implementó una arquitectura cliente-servidor, utilizando FastAPI para el backend, React para el frontend y SQLite como base de datos.

Posteriormente, se desarrolla el sistema mediante la implementación de sus funcionalidades principales, tales como la lógica del chatbot, la interacción con el usuario y la persistencia de información.

Finalmente, se realizan pruebas para validar el funcionamiento del sistema, así como ajustes y mejoras con base en los resultados obtenidos.

Este proceso permite garantizar que las soluciones desarrolladas sean funcionales, eficientes y alineadas a las necesidades institucionales.

5. Estructura Organizacional

El Centro de Cómputo Académico (CECA) cuenta con una estructura organizacional jerárquica que permite una adecuada distribución de funciones y responsabilidades.

En la parte superior se encuentra la Dirección, encargada de la toma de decisiones estratégicas y la supervisión general. De esta dependen la Subdirección, el área de Apoyo Administrativo y el Área de Planeación, las cuales gestionan recursos y coordinan las actividades del centro.

A nivel operativo, el CECA se organiza en distintas áreas y departamentos, entre los que destacan:

- Área de Administración
- Área de Mantenimiento de Equipo de Cómputo
- Departamento de Innovación y Tecnológica Educativa
- Departamento de Infraestructura Tecnológica

Dentro del Departamento de Innovación y Tecnológica Educativa se ubican las áreas de:

- Área de Innovación y Desarrollo
- Área de Tecnología Educativa

El proyecto de estadía fue desarrollado en el Área de Innovación y Desarrollo, la cual se encarga del diseño, desarrollo e implementación de soluciones tecnológicas innovadoras. Esta área trabaja en la creación de sistemas, aplicaciones y herramientas digitales que apoyan el proceso educativo.

Por otro lado, el Departamento de Infraestructura Tecnológica incluye áreas como Control de Usuarios e Infraestructura Tecnológica y Capacitación en Cómputo, las cuales aseguran el correcto funcionamiento de los sistemas y brindan soporte a los usuarios.

La estructura organizacional del CECA permite una coordinación eficiente entre las diferentes áreas, facilitando el desarrollo de proyectos tecnológicos y el cumplimiento de los objetivos institucionales.

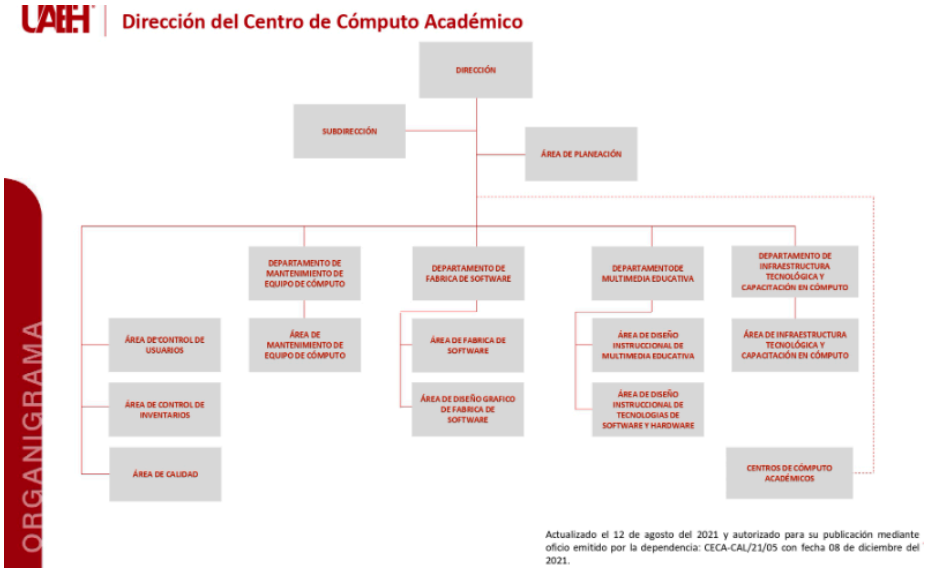


Ilustración 1 Organigrama CeCA

Contexto y Planteamiento del Problema

Diagnóstico

En el contexto educativo actual, el aprendizaje de las matemáticas continúa representando un desafío importante para los estudiantes, especialmente en el área del álgebra. Esta disciplina requiere el desarrollo de habilidades como el razonamiento lógico, la interpretación de símbolos y la aplicación de procedimientos estructurados, lo cual no siempre resulta sencillo para los alumnos de nivel preparatoria.

Dentro del entorno académico de la Universidad Autónoma del Estado de Hidalgo, específicamente en el área de Innovación y Desarrollo del Centro de Cómputo Académico, se identificó la necesidad de implementar soluciones tecnológicas que contribuyan al fortalecimiento del aprendizaje en áreas fundamentales como las matemáticas.

A través de la observación del comportamiento académico de los estudiantes y del análisis de las dificultades más frecuentes, se detecta que muchos alumnos presentan problemas al momento de resolver ejercicios algebraicos. Estas dificultades se relacionan principalmente con la comprensión de los procedimientos y la falta de acompañamiento en el proceso de aprendizaje fuera del aula.

Asimismo, se observa que, aunque existen recursos digitales disponibles, como videos, tutoriales o plataformas educativas, estos no siempre responden de manera específica a las dudas que surgen en el momento. Esto provoca que los estudiantes deban invertir más tiempo en buscar información o, en algunos casos, abandonar el proceso de resolución de problemas.

En este sentido, se identifica un área de oportunidad en el uso de tecnologías emergentes, particularmente la inteligencia artificial, para desarrollar herramientas que permitan brindar apoyo académico de manera inmediata, accesible y personalizada.

Para el análisis y desarrollo del proyecto se utilizaron herramientas tecnológicas como React y Next.js para la construcción de la interfaz de usuario, Node.js y TypeScript para la lógica del sistema, Prisma como gestor de base de datos, y Ollama como motor de inteligencia artificial. Estas herramientas permitieron estructurar el sistema y facilitar su implementación dentro del entorno tecnológico.

Planteamiento del problema

El problema que se aborda en el presente proyecto se centra en la dificultad que presentan los estudiantes de nivel secundaria y preparatoria para comprender y resolver problemas de álgebra, especialmente cuando no cuentan con apoyo académico inmediato fuera del aula.

Actualmente, existe una brecha entre la necesidad de los estudiantes de recibir orientación en el momento en que surgen sus dudas y la disponibilidad de herramientas que les permitan obtener respuestas claras y oportunas. Esta situación genera limitaciones en el proceso de aprendizaje, afectando tanto la comprensión de los temas como el desempeño académico.

La falta de acompañamiento en el estudio independiente provoca que los estudiantes enfrenten obstáculos al intentar resolver ejercicios, lo que puede derivar en errores constantes, pérdida de interés y desmotivación hacia la materia.

Además, aunque existen recursos digitales que ofrecen contenido educativo, estos no siempre permiten una interacción directa ni responden de forma personalizada a las dudas específicas de cada estudiante, lo que limita su efectividad como herramienta de apoyo.

En este sentido, el problema se define como la ausencia de un medio accesible e interactivo que permita a los estudiantes resolver dudas de álgebra en tiempo real, reduciendo la brecha entre la situación actual de aprendizaje y el nivel de comprensión esperado.

Objetivo General

Desarrollar un chatbot académico basado en inteligencia artificial denominado GarzaMath, mediante la implementación de tecnologías de desarrollo web y modelos de lenguaje, para brindar apoyo en la resolución de dudas relacionadas con el álgebra en estudiantes de nivel preparatoria, facilitando la comprensión de conceptos matemáticos y fortaleciendo el aprendizaje autónomo fuera del aula.

Objetivos específicos

• Analizar las necesidades académicas de los estudiantes de secundaria y preparatoria en el área de álgebra para identificar las principales dificultades en su proceso de aprendizaje.

• Diseñar los flujos conversacionales y las preguntas frecuentes que permitan al chatbot interpretar adecuadamente las consultas realizadas por los usuarios.

• Implementar la interfaz del sistema mediante tecnologías web que faciliten la interacción entre el estudiante y el chatbot de forma sencilla e intuitiva.

• Desarrollar la lógica del backend utilizando Node.js y TypeScript para procesar las solicitudes de los usuarios y gestionar la comunicación con el modelo de inteligencia artificial.

• Integrar un modelo de lenguaje que permita generar respuestas claras y comprensibles relacionadas con la resolución de problemas de álgebra.

• Implementar un sistema de almacenamiento de conversaciones que permita registrar las interacciones de los usuarios y mejorar el funcionamiento del chatbot.

• Evaluar el desempeño del sistema mediante pruebas piloto con estudiantes de secundaria y preparatoria para verificar su utilidad como herramienta de apoyo académico.

Hipótesis

Se plantea que el uso de un chatbot académico basado en inteligencia artificial permitirá mejorar el aprendizaje del álgebra en estudiantes de secundaria y preparatoria, debido a que ofrece una interacción inmediata, accesible y personalizada para resolver dudas, en comparación con los métodos tradicionales de estudio que no siempre brindan apoyo en tiempo real.

Justificación

El desarrollo del proyecto GarzaMath surge a partir de la necesidad de mejorar el proceso de aprendizaje del álgebra en estudiantes de nivel secundaria y preparatoria, ya que esta área del conocimiento representa una de las principales dificultades dentro de su formación académica. En muchos casos, los estudiantes presentan problemas para comprender los procedimientos matemáticos, lo que afecta directamente su desempeño y genera desinterés hacia la materia.

Una de las principales problemáticas identificadas es la falta de apoyo inmediato fuera del aula. Cuando los estudiantes realizan actividades de manera independiente, suelen surgir dudas que no pueden resolver en el momento, lo cual provoca que no logren comprender completamente los temas o que cometan errores de forma constante. Esta situación limita el desarrollo de habilidades matemáticas y dificulta su avance académico.

En este contexto, el proyecto GarzaMath se plantea como una alternativa que busca aprovechar el uso de la inteligencia artificial para brindar apoyo académico en tiempo real. A través de un chatbot, los estudiantes pueden realizar preguntas y recibir respuestas de manera inmediata, lo que facilita la comprensión de los procedimientos algebraicos y promueve el aprendizaje autónomo.

La importancia de este proyecto radica en su capacidad para complementar la enseñanza tradicional, ofreciendo una herramienta accesible que puede ser utilizada en cualquier momento y desde cualquier lugar. Esto permite que los estudiantes refuercen sus conocimientos fuera del horario escolar, contribuyendo a mejorar su desempeño académico.

Asimismo, el proyecto beneficia directamente a los estudiantes, quienes obtienen una herramienta de apoyo para resolver dudas; a los docentes, ya que los alumnos pueden llegar con una mejor comprensión de los temas; y a la institución, al integrar el uso de tecnologías innovadoras dentro del proceso educativo.

Desde el punto de vista tecnológico, GarzaMath también representa una oportunidad para aplicar conocimientos en desarrollo web e inteligencia artificial, lo que contribuye al fortalecimiento de competencias en el área de innovación y desarrollo.

En conclusión, el proyecto se justifica por su utilidad como herramienta de apoyo académico, su contribución al aprendizaje de los estudiantes y su potencial para integrar tecnologías emergentes en el ámbito educativo, ofreciendo una solución accesible, innovadora y alineada con las necesidades actuales.

Marco teórico

Introducción

En la última década, el avance de las tecnologías de la información ha transformado de manera significativa los procesos de enseñanza-aprendizaje, dando lugar al desarrollo de herramientas digitales que facilitan el acceso al conocimiento y promueven nuevas formas de interacción entre los estudiantes y los contenidos educativos. En particular, la incorporación de sistemas basados en inteligencia artificial ha permitido la creación de entornos de aprendizaje más dinámicos, personalizados e interactivos.

Dentro de este contexto, el uso de herramientas tecnológicas en la educación se ha convertido en un elemento fundamental para apoyar el aprendizaje, especialmente en áreas como las matemáticas, donde los estudiantes suelen presentar mayores dificultades. El álgebra, al ser una de las ramas fundamentales de esta disciplina, requiere el desarrollo de habilidades como el razonamiento lógico, la interpretación simbólica y la resolución estructurada de problemas, lo cual puede representar un reto para estudiantes de nivel medio superior y superior.

Ante esta problemática, surge la necesidad de implementar soluciones tecnológicas que permitan brindar apoyo académico de manera accesible, interactiva y personalizada. En este sentido, los chatbots educativos basados en inteligencia artificial representan una alternativa innovadora que puede contribuir a mejorar la comprensión de los contenidos matemáticos, facilitando el aprendizaje autónomo y la resolución de dudas en tiempo real.

Antecedentes del tema

En los últimos años, diversas instituciones educativas han incorporado tecnologías digitales en sus procesos de enseñanza con el objetivo de mejorar la calidad educativa y optimizar el aprendizaje de los estudiantes. Entre estas tecnologías destacan los sistemas basados en inteligencia artificial, los cuales han permitido el desarrollo de plataformas educativas, asistentes virtuales y sistemas de tutoría automatizada.

Los chatbots educativos han emergido como herramientas relevantes dentro de este ámbito, ya que permiten interactuar con los usuarios mediante lenguaje natural y ofrecer respuestas automatizadas a sus consultas. De acuerdo con Winkler y Söllner (2018), estos sistemas pueden mejorar la experiencia de aprendizaje al proporcionar retroalimentación inmediata y facilitar la interacción continua entre el estudiante y el sistema.

En el área de las matemáticas, se han desarrollado diversas plataformas digitales que buscan facilitar la comprensión de los contenidos mediante recursos interactivos, como ejercicios dinámicos, simuladores y videos explicativos. Sin embargo, muchas de estas herramientas presentan limitaciones, ya que no ofrecen una interacción personalizada ni la posibilidad de resolver dudas específicas en tiempo real (Holmes et al., 2019).

En este contexto, el desarrollo de GarzaMath se plantea como una solución tecnológica que integra inteligencia artificial y desarrollo web para crear un chatbot educativo enfocado en el aprendizaje del álgebra, dirigido a estudiantes de bachillerato y nivel superior.

A partir de los antecedentes ya presentados, se establecen las bases teóricas que sustentan el desarrollo del proyecto GarzaMath.

Bases teóricas

Inteligencia Artificial.

La inteligencia artificial (IA) es una rama de la informática que se enfoca en el desarrollo de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, el razonamiento y la toma de decisiones. Según Russell y Norvig (2021), la IA puede definirse como el estudio de agentes que perciben su entorno y toman acciones para maximizar el logro de sus objetivos.

En el ámbito educativo, la IA ha permitido el desarrollo de sistemas adaptativos que personalizan el contenido en función de las necesidades del estudiante, mejorando la eficiencia del proceso de aprendizaje. En el caso de los chatbots educativos, la IA permite interpretar las consultas de los usuarios y generar respuestas coherentes, facilitando la interacción entre el estudiante y el sistema.

El procesamiento del lenguaje natural.

El Procesamiento de Lenguaje Natural (PLN) es una subárea de la inteligencia artificial que se encarga de la interacción entre las computadoras y el lenguaje humano. Su objetivo es permitir que los sistemas comprendan, interpreten y generen lenguaje natural de manera efectiva (Jurafsky & Martin, 2020).

El PLN integra diversas técnicas, tales como la tokenización, el análisis sintáctico y la identificación de la intención del usuario. Estas técnicas son fundamentales en el desarrollo de chatbots, ya que permiten analizar las preguntas formuladas por los usuarios y generar respuestas adecuadas.

En el caso de GarzaMath, el PLN se emplea para interpretar preguntas relacionadas con álgebra y proporcionar explicaciones comprensibles, facilitando así el aprendizaje del estudiante.

Chatbots educativos.

Un chatbot es un programa informático diseñado para simular conversaciones con usuarios mediante texto o voz. En el ámbito educativo, los chatbots se utilizan como herramientas de apoyo que permiten resolver dudas, proporcionar información y guiar a los estudiantes en su proceso de aprendizaje.

Los chatbots educativos ofrecen diversas ventajas, entre las que destacan su disponibilidad continua, la interacción en tiempo real y la capacidad de adaptarse a diferentes usuarios. Estas características los convierten en una herramienta eficaz para complementar el aprendizaje tradicional (Winkler & Söllner, 2018).

GarzaMath se clasifica como un chatbot educativo, ya que su objetivo es brindar apoyo académico en el área de álgebra mediante una interfaz conversacional que facilita la interacción entre el usuario y el sistema.

Aprendizaje autónomo

El aprendizaje autónomo se refiere a la capacidad del estudiante para gestionar su propio proceso de aprendizaje, identificando sus necesidades y utilizando recursos para adquirir conocimientos de manera independiente (Zimmerman, 2002).

El uso de herramientas tecnológicas como los chatbots educativos favorece este tipo de aprendizaje, ya que permite a los estudiantes acceder a información en cualquier momento, resolver dudas y reforzar conocimientos sin depender exclusivamente del docente.

Enseñanza del álgebra

El álgebra es una rama fundamental de las matemáticas que se centra en el uso de símbolos y expresiones para representar relaciones y resolver problemas. Su enseñanza es clave en niveles de educación media superior y superior, ya que constituye la base para el desarrollo de habilidades matemáticas más avanzadas.

Sin embargo, el aprendizaje del álgebra suele representar un desafío debido a la abstracción de sus conceptos. Por ello, es importante contar con herramientas que faciliten su comprensión mediante explicaciones claras, ejemplos prácticos y apoyo continuo (Kieran, 2007).

Tecnologías de desarrollo web

El desarrollo de aplicaciones web permite la creación de sistemas accesibles desde múltiples dispositivos con conexión a internet. Estas aplicaciones se basan generalmente en una arquitectura cliente-servidor, donde el frontend gestiona la interfaz de usuario y el backend procesa la lógica del sistema. En el caso de GarzaMath, se emplea React para el desarrollo de la interfaz de usuario, permitiendo una interacción dinámica e intuitiva. Por su parte, el backend se implementa con FastAPI, el cual permite gestionar las solicitudes del usuario mediante una API REST.

Para la gestión de datos, se utiliza Prisma como ORM (Object-Relational Mapping), lo que permite interactuar de manera eficiente con la base de datos. Esta herramienta facilita el almacenamiento, consulta y manejo de los historiales de conversación generados por el chatbot, permitiendo una gestión estructurada y escalable de la información.

El uso conjunto de estas tecnologías permite desarrollar un sistema robusto, eficiente y orientado a mejorar la experiencia del usuario.

Relación con el proyecto

El conjunto de conceptos teóricos presentados sustenta el desarrollo de GarzaMath, ya que integra elementos de inteligencia artificial, procesamiento de lenguaje natural y desarrollo web para la creación de un chatbot educativo enfocado en el aprendizaje del álgebra.

Asimismo, el uso de tecnologías como FastAPI, React y Prisma permite implementar una solución funcional que facilita la interacción con el usuario y el almacenamiento de historiales de conversación, contribuyendo al aprendizaje autónomo y al fortalecimiento de habilidades matemáticas en los estudiantes.

Metodología

Para el desarrollo del presente proyecto se adoptó un enfoque basado en metodologías ágiles, tomando como referencia el marco de trabajo Scrum, el cual fue adaptado a las condiciones reales del entorno de estadía. Esta metodología permitió organizar el trabajo de manera flexible, favoreciendo la implementación progresiva del sistema y la mejora continua de sus funcionalidades.

A diferencia de metodologías tradicionales, el enfoque ágil permitió trabajar por etapas cortas, enfocándose en la entrega constante de avances funcionales. Esto resultó especialmente útil debido a la naturaleza del proyecto, ya que el desarrollo del chatbot implicó la integración de distintos componentes, así como la necesidad de realizar pruebas continuas y ajustes durante el proceso. Cabe mencionar que no se implementó Scrum de manera estricta (con todos sus roles y eventos formales), sino que se adoptaron sus principios fundamentales, tales como el desarrollo iterativo, la priorización de tareas mediante un Product Backlog y la organización del trabajo en Sprints. Esto permitió mantener un equilibrio entre estructura y flexibilidad.

Secuencia Metodológica

El desarrollo del sistema se llevó a cabo mediante una secuencia lógica de actividades, las cuales permitieron dar solución a la problemática planteada. Estas etapas se desarrollaron de manera iterativa, repitiéndose cuando fue necesario para mejorar el sistema.

- **Planeación:** En esta etapa se definieron los objetivos del proyecto, así como el alcance del chatbot académico. Se identificaron las principales funcionalidades que debía cumplir el sistema, considerando las necesidades de los usuarios y los recursos disponibles.
- **Análisis de requerimientos:** Se identificaron los requerimientos funcionales del sistema, tales como la interacción mediante chat, la resolución de ejercicios de álgebra, el almacenamiento de conversaciones y la gestión de usuarios. Esta etapa permitió establecer una base clara para el desarrollo.
- **Diseño del sistema:** Se definió la arquitectura del sistema bajo un modelo cliente-servidor. Asimismo, se seleccionaron las tecnologías a utilizar: FastAPI para el backend, React (Vite) para el frontend y SQLite como sistema de almacenamiento.

  También se diseñó la estructura del sistema, incluyendo endpoints, flujo de datos y organización del código.
- **Desarrollo (iteraciones ágiles):** El desarrollo se llevó a cabo de forma incremental, implementando las funcionalidades del sistema por partes. Cada componente fue desarrollado, probado y ajustado antes de continuar con el siguiente.

  Durante esta etapa se implementan funcionalidades como:
  - Comunicación mediante el endpoint /chat
  - Gestión de estudiantes y conversaciones
  - Persistencia de mensajes en base de datos
  - Lógica del chatbot académico

  Este proceso se repitió en varias ocasiones, permitiendo mejorar continuamente el sistema.

- **Pruebas:** Se realizaron pruebas de funcionamiento para verificar la correcta operación del sistema. En esta etapa se detectaron problemas como errores de integración y tiempos de respuesta elevados (timeout), los cuales fueron corregidos en iteraciones posteriores.
- **Implementación y mejora continua:** Finalmente, se realizaron ajustes y optimizaciones al sistema con base en los resultados obtenidos. Se mejoró el rendimiento, la estabilidad y la calidad de las respuestas del chatbot.

  Cabe destacar que, al tratarse de un enfoque ágil, el sistema se encuentra en constante posibilidad de mejora.

Product backlog

Como parte de la metodología ágil, se utilizó un Product Backlog para organizar y priorizar las funcionalidades del sistema. Este backlog incluyó historias de usuario que describen las necesidades del sistema desde la perspectiva del usuario final.

Las tareas se clasificaron de acuerdo con su prioridad (alta, media y baja), permitiendo enfocar el desarrollo en las funcionalidades más importantes, como la interacción con el chatbot, el procesamiento de mensajes y la persistencia de datos.

El Product Backlog sirvió como base para la planificación del desarrollo y permitió mantener una visión clara del progreso del proyecto.

ID	Historia de Usuario	Descripción	Prioridad	Esfuerzo	Estado
1	Como estudiante, quiero interactuar con el chatbot	Permitir que el usuario envíe mensajes y reciba respuestas	Alta	1 semana	Completado
2	Como sistema, quiero procesar mensajes	Interpretar las entradas del usuario y generar respuestas	Alta	1 semana	Completado
3	Como estudiante, quiero resolver ejercicios de álgebra	Resolver ecuaciones básicas paso a paso	Alta	1 semana	Completado
4	Como sistema, quiero almacenar conversaciones	Guardar historial de mensajes en base de datos	Alta	1 semana	Completado
5	Como sistema, quiero identificar usuarios	Asociar conversaciones a un estudiante	Alta	1 semana	Completado
6	Como estudiante, quiero ver mi historial	Consultar conversaciones anteriores	Media	1 semana	Completado
7	Como sistema, quiero manejar errores	Evitar fallos en respuestas o caídas del sistema	Alta	2 semanas	En proceso
8	Como sistema, quiero optimizar tiempos de respuesta	Reducir problemas de timeout	Alta	1 semana	En proceso
9	Como estudiante, quiero explicaciones claras	Obtener respuestas paso a paso	Alta	1 semana	Completado
10	Como sistema, quiero integrar lógica de tutor	Detectar si el usuario está confundido	Media	1 semana	En proceso
11	Como estudiante, quiero interfaz amigable	Facilitar la interacción en el frontend	Media	1 semana	Completado
12	Como sistema, quiero conectar frontend y backend	Comunicación mediante API REST	Alta	1 semana	Completado
13	Como sistema, quiero persistencia de datos	Uso de SQLite para almacenamiento	Alta	1 semana	Completado
14	Como sistema, quiero escalar a futuro	Preparar integración con IA (LLM)	Baja	Pendiente	Pendiente
15	Como administrador, quiero monitorear el sistema	Registrar logs de interacción	Media	Pendiente	Pendiente

Ilustración 2 Tabla Product Backlog

Sprint Backlog

El desarrollo del proyecto se organizó en Sprint, lo que permitió avanzar de manera ordenada en la implementación del chatbot académico.

- Sprint 1: Configuración base

  Se configuró el backend con FastAPI, se creó el endpoint /chat y se integró Prisma como ORM para la gestión de la base de datos.

  Resultado: Sistema base funcional con persistencia de datos.

- Sprint 2: Funcionalidades principales

  Se implementó la resolución de ejercicios de álgebra, el almacenamiento de conversaciones mediante Prisma y la interfaz en React.

  Resultado: Chatbot funcional con interacción completa.

- Sprint 3: Optimización

  Se mejoró el rendimiento del sistema, se corrigieron errores y se redujeron problemas de tiempo de respuesta (timeout).

  Resultado: Sistema más estable y eficiente.

- Sprint 4: Mejoras futuras

  Se contemplan mejoras como la integración con modelos de inteligencia artificial, sistema de logs y optimización de la experiencia de usuario.

  Resultado: Proyecto preparado para escalabilidad.

Representación de la metodología

La metodología empleada puede representarse como un ciclo iterativo en el cual las etapas se repiten con el objetivo de mejorar el sistema:

Ilustración 3 Metodología

Justificación de la metodología

El uso e implementación de una metodología ágil permitió desarrollar el sistema de manera flexible, facilitando la adaptación a cambios durante el proceso y la incorporación de nuevas funcionalidades.

Además, el enfoque iterativo favoreció la detección temprana de errores y la mejora continua del sistema, lo cual fue fundamental para lograr un producto funcional. Asimismo, el uso de herramientas como el Product Backlog y los Sprints permitió organizar el trabajo de manera eficiente.

Para finalizar, la metodología empleada contribuyó significativamente al desarrollo del chatbot académico, permitiendo integrar sus funcionalidades de forma progresiva y alineada a los objetivos del proyecto.

Desarrollo del Proyecto

El desarrollo del sistema GarzaMath se llevó a cabo mediante la implementación de una arquitectura web moderna basada en el modelo cliente-servidor, integrando tecnologías actuales que permiten construir aplicaciones escalables, mantenibles y eficientes.

El sistema está diseñado como una aplicación web interactiva que permite a los usuarios comunicarse con un asistente académico basado en inteligencia artificial, el cual procesa consultas relacionadas con álgebra y genera respuestas en tiempo real.

Para lograr este objetivo, se integraron múltiples tecnologías que trabajan de manera conjunta:

- Next.js (framework fullstack)
- React (interfaz de usuario)
- API Routes (backend)

- Prisma (ORM)
- SQLite (base de datos)
- Ollama (modelo de inteligencia artificial local)

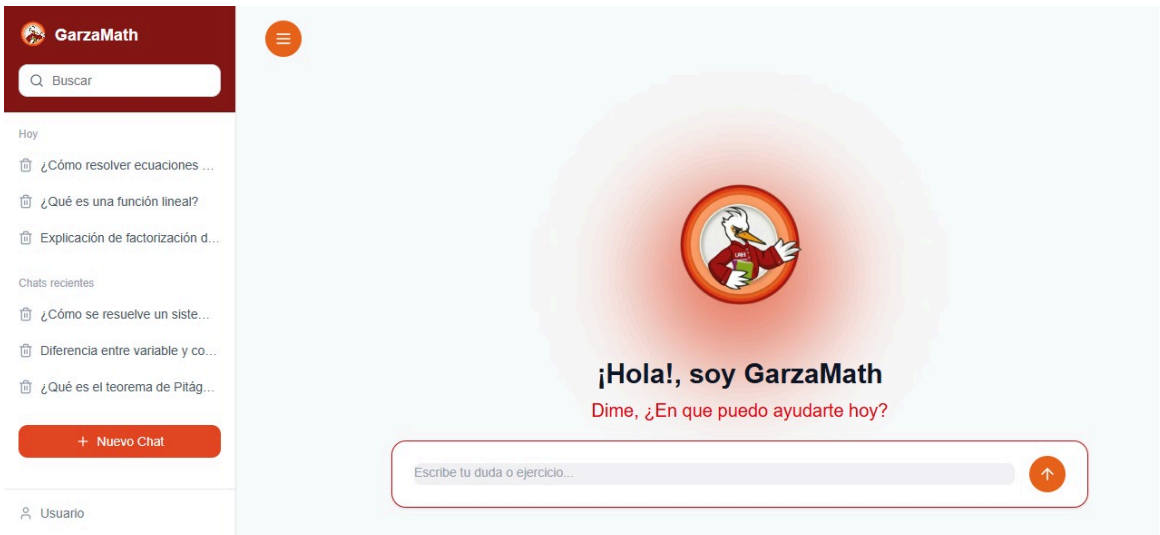


Ilustración 4 Interfaz final de GarzaMath

Instalación y configuración inicial del entorno de desarrollo

El desarrollo del sistema GarzaMath comenzó con la preparación del entorno de trabajo, etapa fundamental para asegurar el correcto funcionamiento de las tecnologías que se utilizarían a lo largo del proyecto. En esta fase se definieron las herramientas base, así como la estructura inicial del sistema.

En primer lugar, se realizó la instalación de Node.js, el cual es necesario para ejecutar aplicaciones basadas en JavaScript tanto en el cliente como en el servidor. Esta herramienta permitió utilizar npm (Node Package Manager), que se empleó para la gestión de dependencias del proyecto.

Una vez configurado el entorno, se procedió a la creación del proyecto utilizando Next.js, un framework que integra React y facilita el desarrollo de aplicaciones web modernas. La elección de Next.js se debió a que permite trabajar en un entorno fullstack, es decir, desarrollar tanto la interfaz de usuario como la lógica del servidor dentro del mismo proyecto.

La creación del proyecto se llevó a cabo mediante el siguiente comando:

npx create-next-app@latest algebra-bot

Durante este proceso se seleccionaron configuraciones como el uso de TypeScript, lo cual permitió mejorar la estructura del código, así como el uso de App Router, que facilita la organización de rutas dentro del sistema.

Una vez creado el proyecto, se accedió a la carpeta correspondiente y se ejecutó en modo desarrollo:

```
Cd      algebra-bot
npm run dev
```

Con esto, se verificó que el sistema se ejecute correctamente en el entorno local, asegurando que la base del proyecto estuviera lista para continuar con el desarrollo

Estructura real del proyecto

Una vez generado el proyecto, se trabajó sobre la estructura que Next.js crea por defecto, adaptándola a las necesidades del sistema. La organización final del proyecto quedó de la siguiente manera:

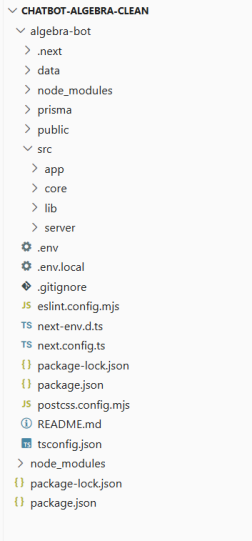


Ilustración 5 Organización del proyecto

Organización interna del sistema (arquitectura limpia)

Con el objetivo de mantener una arquitectura limpia, el desarrollo principal del sistema se concentró dentro de la carpeta src, donde se organizaron los componentes de acuerdo con su función.

Esta decisión permitió separar claramente las responsabilidades del sistema, evitando mezclar lógica de negocio con la interfaz o configuraciones.

Dentro de src, el proyecto se organizó de la siguiente forma conceptual:

- **src/app** Contiene las vistas y rutas del sistema (frontend)
- **src/app/api** Define las rutas del backend (API Routes)
- **src/lib** Configuración de servicios como Prisma
- **src/server / core (según implementación)** Lógica del sistema

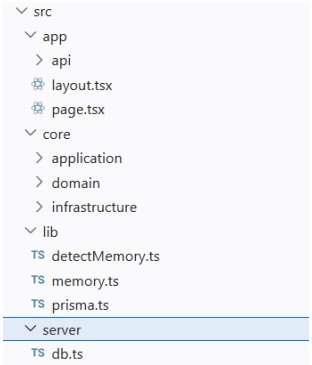


Ilustración 6 Arquitectura

Descripción de carpetas principales del proyecto

Durante el desarrollo, cada carpeta cumplió una función específica:

- **algebra-bot/**

Contiene la aplicación principal donde se desarrolló todo el sistema.

- **.next/**

Carpeta generada automáticamente por Next.js donde se almacenan los archivos compilados del proyecto. No se modifica manualmente.

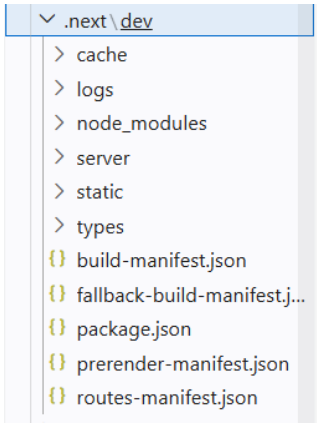


Ilustración 7 Carpeta .next

- **data/**

Utilizada para almacenar información auxiliar o archivos generados durante el desarrollo. En este proyecto puede emplearse para pruebas o datos temporales.

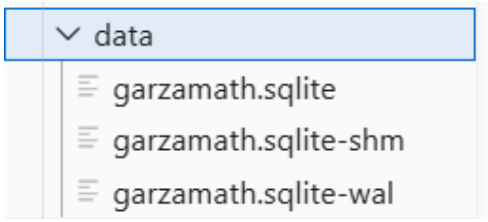


Ilustración 8 sección data

- **node_modules/**

Contiene todas las dependencias instaladas mediante npm. Es fundamental para el funcionamiento del sistema, pero no se modifica manualmente.

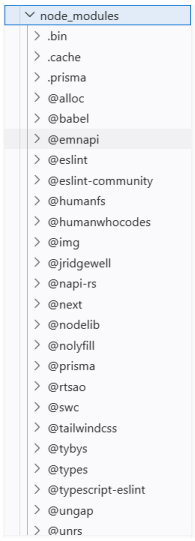


Ilustración 9 Dependencias Instaladas

- **prisma/**

Contiene la configuración de la base de datos, incluyendo el archivo schema.prisma, donde se define el modelo de datos utilizado en el sistema.

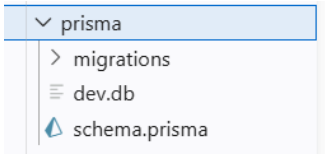


Ilustración 10 Archivo prisma

- **public/**

Almacena recursos estáticos como imágenes, íconos o archivos accesibles desde el navegador.

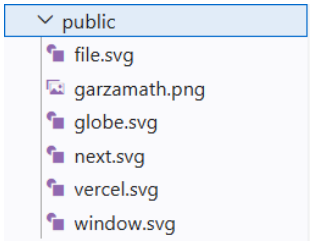


Ilustración 11 Ruta public

- **src/**

Es la carpeta principal donde se desarrolla el sistema. Aquí se encuentra la lógica del frontend, backend y configuraciones internas.

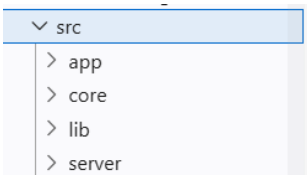


Ilustración 12 Carpeta src

- **.env y .env.local**

Archivos utilizados para definir variables de entorno, como la configuración del modelo de inteligencia artificial o la base de datos.

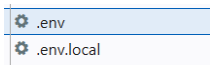


Ilustración 13 variables de entorno

- **package.json**

Archivo que contiene las dependencias del proyecto y los scripts necesarios para su ejecución.

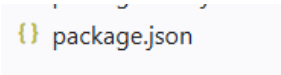


Ilustración 14 Package json

- **tsconfig.json**

Archivo de configuración de TypeScript que permite definir reglas para el desarrollo del sistema.

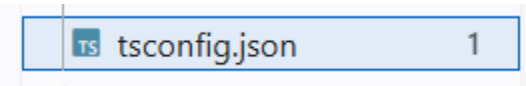


Ilustración 15 Archivo que contiene la configuración de typeScript

Preparación del sistema para desarrollo

Una vez definida la estructura del proyecto, se procedió a verificar que todas las dependencias estuvieran correctamente instaladas y que el sistema pudiera ejecutarse sin errores.

Durante esta etapa, se realizaron pruebas iniciales ejecutando el servidor en modo desarrollo, validando que la estructura implementada no afectará el funcionamiento del sistema.

Esta fase fue importante, ya que permitió establecer una base sólida para continuar con el desarrollo del frontend, el backend y la integración de la inteligencia artificial.

Descripción funcional de archivos dentro de la carpeta src

Después se procedió a desarrollar cada uno de los componentes del sistema dentro de la carpeta src, implementando archivos específicos que permiten el funcionamiento del chatbot.

A continuación, se describen los archivos principales desarrollados y su función dentro del sistema.

Archivo: src/app/api/chat/route.ts

Este archivo fue uno de los más importantes durante el desarrollo, ya que aquí se implementa la lógica principal para procesar los mensajes del usuario.

Básicamente, este archivo funciona como el “puente” entre la interfaz del chat y el modelo de inteligencia artificial. Cada vez que el usuario escribe un mensaje, este archivo es el encargado de recibirlo y decidir qué hacer con él.

Ilustración 16 archivo src/app/api/chat/route.ts

Archivo: src/app/api/conversations/route.ts

Este archivo se creó después, cuando se vio la necesidad de guardar las conversaciones del usuario.

Al principio el sistema solo respondía, pero no guardaba nada. Entonces se implementó esta ruta para poder consultar el historial de chats.

Lo que hace este archivo es conectarse a la base de datos y devolver todas las conversaciones que se han guardado. Esto permite que el usuario pueda ver interacciones anteriores.

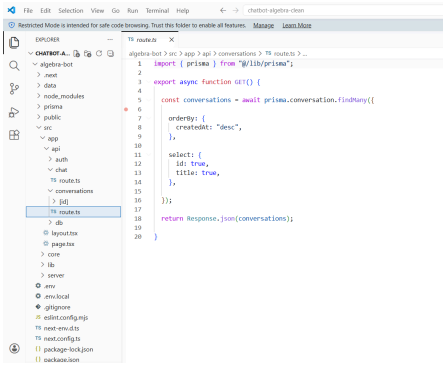


Ilustración 17 Conversations

Archivo: src/app/chat/page.tsx

Este archivo corresponde a la parte visual del sistema, es decir, donde el usuario interactúa con el chatbot.

Aquí fue donde se desarrolló toda la interfaz tipo chat. Primero se implementó algo muy básico para mostrar mensajes, y poco a poco se fue mejorando.

Para manejar los mensajes se utilizó un estado:

```
const [messages, setMessages] = useState([]);
```

Esto permitió que cada vez que el usuario escribiera algo, el mensaje se agregara a la conversación y se muestre en pantalla.

También se implementó la función para enviar mensajes al backend:

```
Await  fetch("/api/chat"{
 method:"POST",
 body:JSON.stringify({message:input}),
});
```

Archivo: src/lib/prisma.ts

Este archivo se creó para manejar la conexión con la base de datos de una forma más organizada.

En lugar de estar creando conexiones en cada archivo, se decidió centralizar todo en uno solo, lo cual facilitó mucho el desarrollo.

Aquí básicamente se inicializa Prisma:

```
import{PrismaClient}from"@prisma/client";

export const prisma = new PrismaClient();
```

Archivo: prisma/schema.prisma

En este archivo se definió cómo se iban a guardar los datos en la base de datos.

Durante el desarrollo, se decidió guardar las conversaciones en un solo campo como texto, lo cual simplificó la implementación.

```
Model Conversation{
 idString@id@default(cuid())
messagesString
createdAtDateTime@default(now())
}
```

Archivo: .env

Este archivo se utilizó para guardar configuraciones importantes, principalmente las relacionadas con la inteligencia artificial.

Por ejemplo, aquí se definió la URL de Ollama y el modelo que se VA a utilizar:

```
OLLAMA_URL=http://127.0.0.1:11434
OLLAMA_MODEL=qwen2.5:1.5b
```

Archivo: package. json

Este archivo ya viene con el proyecto, pero fue muy importante durante el desarrollo porque aquí se manejan las dependencias y los comandos para ejecutar el sistema.

Desarrollo del frontend (interfaz del chatbot)

El primer paso fue crear la vista principal del chatbot dentro del archivo:

src/app/chat/page.tsx

En este archivo se comenzó a implementar una estructura básica que permitiera mostrar mensajes en pantalla y capturar la entrada del usuario.

Al inicio, se trabajó con una estructura simple, donde únicamente se mostraba texto, y poco a poco se fue adaptando para simular una conversación.

Manejo de mensajes en la interfaz

Para poder gestionar la conversación, se utilizó el hook **useState**, el cual permite almacenar los mensajes en memoria dentro del frontend.

Cada mensaje contiene información básica como:

- Quién lo envía (usuario o asistente)
- El contenido del mensaje

Esto permitió que, cada vez que el usuario escribiera algo, el mensaje se agregara automáticamente a la lista y se mostrará en pantalla.

```
const [messages, setMessages] = useState([]);
```

Captura de entrada del usuario

Después de poder mostrar mensajes, el siguiente paso fue permitir que el usuario escribiera dentro de la interfaz.

Para ello, se implementó un campo de entrada (input) controlado mediante estado, este valor se actualiza conforme el usuario escribe, lo que permite capturar el mensaje antes de enviarlo.

```
const [input, setInput] = useState("");
```

Actualización dinámica de la conversación

Una vez recibida la respuesta del backend, se actualiza el estado de los mensajes para mostrar tanto el mensaje del usuario como la respuesta del chatbot.

Esto se logró agregando nuevos elementos al arreglo de mensajes:

```
setMessages((prev) => [...prev, newMessage]);
```

Renderizado de los mensajes

Para mostrar los mensajes en pantalla, se utilizó un método de iteración que recorre todos los mensajes almacenados:

```
messages.map((msg,index)=>(
<pkey={index}>{msg.content}</p>
));
```

Integración completa del frontend

Una vez implementadas todas estas funcionalidades, el frontend quedó completamente integrado con el backend.

El flujo dentro de la interfaz funciona de la siguiente manera:

- El usuario escribe un mensaje
- El mensaje se muestra en pantalla
- Se envía al backend
- Se recibe una respuesta
- La respuesta se muestra automáticamente

Desarrollo del backend (procesamiento del chatbot)

Manejo de peticiones HTTP

Dentro del archivo **route.ts**, una parte importante fue definir cómo se maneja la petición POST. Esta línea define que el backend va a responder cuando el frontend haga una petición tipo POST. Esto fue importante porque todas las preguntas del usuario se envían usando este tipo de petición.

```
export async function POST(req: Request) {
```

Procesamiento del mensaje antes de enviarlo a la IA

Antes de mandar el mensaje a Ollama, se puede preparar o controlar el contenido.

```
const userMessage = message.trim();
```

Estructura del prompt (comportamiento del chatbot)

Una parte muy importante del backend fue definir **cómo se le habla al modelo**.

```
messages: [
  {
    role: "system",
    content: "Eres GarzaMath, un tutor de álgebra que explica paso a paso"
  },
  {
    role: "user",
    content: message
  }
]
```

Manejo de errores

Durante el desarrollo, fue necesario evitar que el sistema se rompiera si algo fallaba.

```
try {
  // lógica del chatbot
} catch (error) {
  return Response.json({ error: "Error en el servidor" });
}
```

Guardado de mensajes en base de datos

Cuando se integró Prisma, se añadió la lógica para guardar datos.

```
await prisma.conversation.create({
  data: {
    messages: message
  }
});
```

Consulta de conversaciones

En el archivo de conversaciones, se implementó algo como:

```
const data = await prisma.conversation.findMany({
  orderBy: { createdAt: "desc" }
});
```

Uso de variables de entorno (.env)

Otra parte importante fue no dejar valores fijos en el código.

```
const model = process.env.OLLAMA_MODEL;
```

Decisiones importantes durante el desarrollo

Durante el desarrollo del backend, se tomaron varias decisiones importantes. Una de ellas fue utilizar Ollama en lugar de una API externa, lo cual permitió trabajar de manera local y tener mayor control sobre el sistema. También se decidió utilizar una estructura simple en la base de datos para evitar complejidad innecesaria en etapas iniciales.

Otra decisión relevante fue integrar el backend dentro de Next.js en lugar de separarlo, lo cual simplificó la comunicación entre componentes y permitió un desarrollo más rápido.

Ilustración 18 Logo OLLAMA

Diseños y maquetación de interfaz

Primero se diseñó una maquetación en Figma usando una paleta de colores que corresponden a la institución y se usó la IA que implementa Figma para llegar a un diseño final y ser aprobado por los encargados del área.

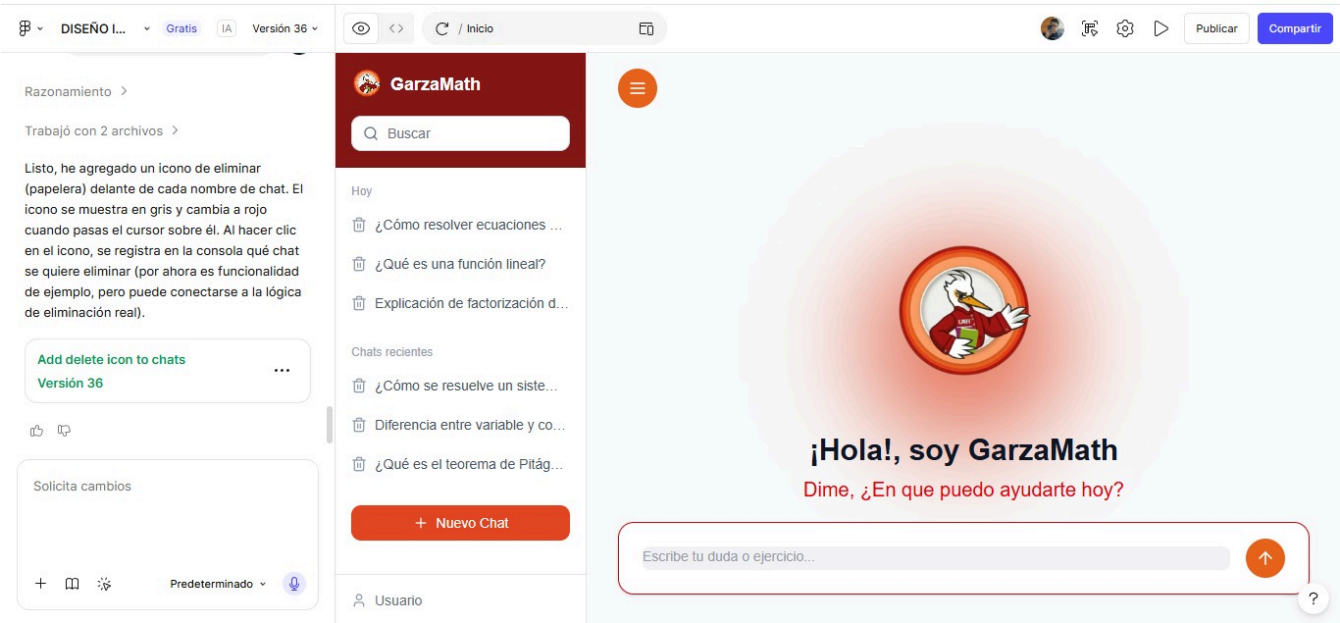


Ilustración 19 Maquetación en plataforma Figma

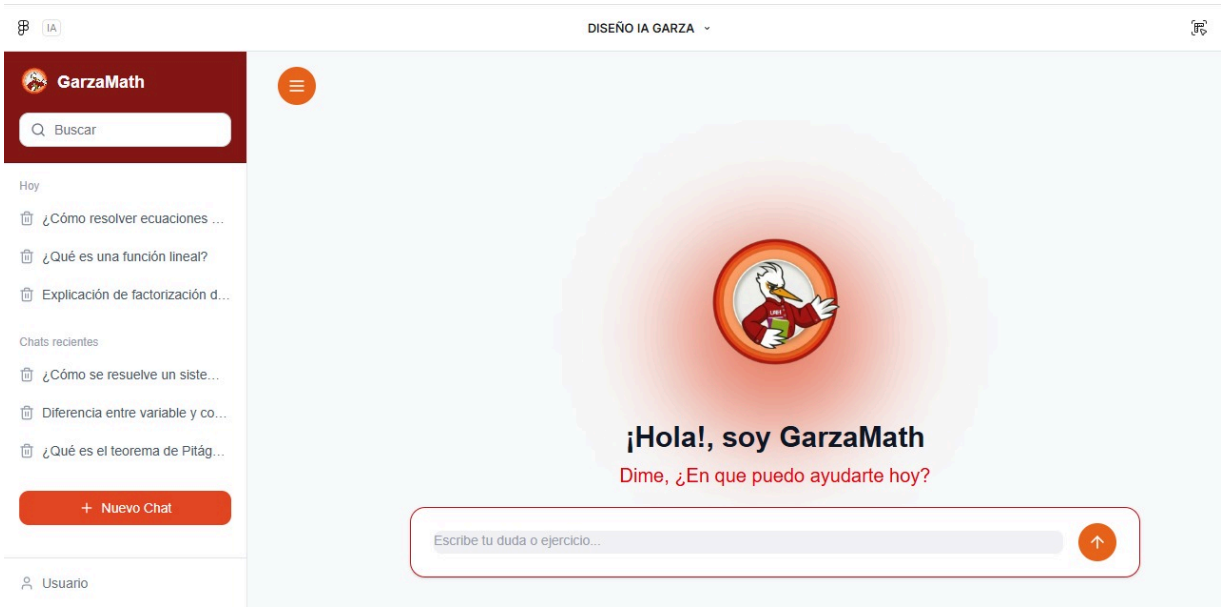


Ilustración 20 diseño final garzamath

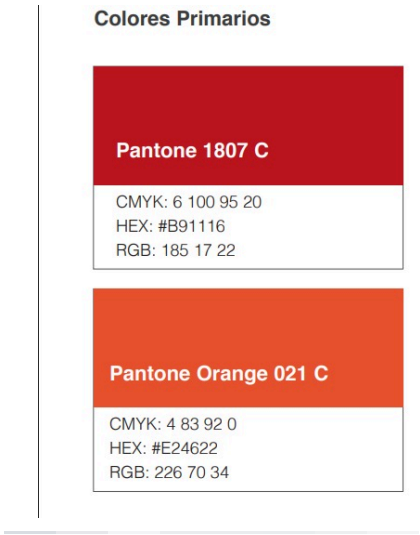


Ilustración 21 Paleta de colores utilizada para diseño. Colores institucionales UAEH.

Problemas y errores durante el desarrollo

- **Error: comunicación frontend – backend (/api/chat)**

Uno de los primeros errores ocurrió al momento de enviar mensajes desde el frontend hacia el backend.

```
await fetch("/api/chat", {
  method: "POST",
  body: JSON.stringify({ message: input }),
});
```

Problema:

El backend no recibía correctamente el mensaje o la petición fallaba.

Solución:

Se corrigió la estructura del fetch, asegurando que los datos se enviaran correctamente en formato JSON y que la ruta estuviera bien definida.

- **Error: mensajes no se mostraban en pantalla**

  Relacionado con el archivo page.tsx

```
setMessages((prev) => [...prev, newMessage]);
```

Problema:

Los mensajes no se actualizaban o no aparecían en el chat.

Solución:

Se corrigió el manejo del estado con useState, asegurando que los mensajes se agregan correctamente al arreglo.

- **Error: lentitud en respuestas (IA)**

  Relacionado con Ollama

Problema:

El chatbot tardaba demasiado en responder o parecía que no respondía.

Solución:

- Se ajustó el modelo utilizado
- Se mejoró la forma de enviar las peticiones
- Se optimizó el flujo del backend

- **Error: problemas con la base de datos**

  Relacionado con Prisma y SQLite

```
await prisma.conversation.create({
  data: { messages: message },
});
```

Problema:

No se guardaban las conversaciones o había errores en la conexión.

Solución:

- Se revisó el archivo schema.prisma
- Se corrigió la estructura del modelo
- Se verificó la conexión con la base de datos

- **Error: pérdida de contexto del chatbot**

  Relacionado con la estructura del prompt

```
messages: [{ role: "user", content: message }]
```

Problema:

El chatbot respondía como si no recordara la conversación.

Solución:

Se mejoró la estructura del prompt agregando contexto e instrucciones.

- **Error: problemas de interfaz (UI)**

Problema:

- Estilos que rompían el diseño
- Elementos visuales confusos

Solución:

- Se eliminaron estilos innecesarios
- Se simplificó la interfaz

Evaluación del proyecto

La evaluación del proyecto GarzaMath se llevó a cabo con la intención de analizar si realmente vale la pena desde distintos ángulos. No solo se revisó cómo funciona técnicamente, sino también el impacto que puede tener en el ámbito educativo y qué recursos se necesitan para ponerlo en marcha. Esta parte es clave, porque ayuda a entender si el sistema cumple con su propósito principal: apoyar a los estudiantes en el aprendizaje del álgebra, y si tiene futuro para seguir mejorando.

Durante el desarrollo del proyecto, se detectaron varios factores que influyen en su funcionamiento. Por eso, la evaluación se organizó en tres áreas principales: social y económica, técnica y financiera. Esto permitió tener una visión más completa de qué tan útil puede ser en la vida real.

Evaluación social y económica

Desde el lado social, el proyecto aporta algo positivo, ya que busca resolver un problema real: a muchos estudiantes se les dificulta el álgebra, sobre todo cuando estudian solos y no tienen quién les explique al momento. Aquí es donde GarzaMath entra como apoyo, ofreciendo respuestas en tiempo real mediante un chatbot.

Una de sus mayores ventajas es que es accesible. Al ser una aplicación web, se puede usar desde cualquier dispositivo con internet. Esto permite que los estudiantes resuelvan dudas en cualquier momento, sin depender completamente de un maestro o del salón de clases, fomentando así el aprendizaje por cuenta propia.

Además, el uso de un chatbot hace que todo sea más sencillo. En lugar de buscar información en distintos lados, el usuario solo hace una pregunta y recibe una respuesta directa. Esto ahorra tiempo y evita la frustración que muchos sienten cuando no entienden un tema.

En cuanto a lo económico, también tiene puntos fuertes. Se utilizaron herramientas gratuitas y de código abierto, lo que ayudó a reducir costos. Tecnologías como Next.js, React, Prisma y SQLite no requieren pagar licencias, lo cual facilita que escuelas o incluso estudiantes puedan usar el sistema sin problema.

Otro punto importante es que el modelo de inteligencia artificial funciona de manera local con Ollama, lo que evita gastos por servicios externos. Es decir, no hay que pagar por cada consulta que se haga, lo cual representa un gran ahorro.

En general, en este aspecto, el proyecto logra ser útil, accesible y económico.

Evaluación técnica

En la parte técnica, el proyecto cumplió con lo que se había planeado desde el inicio. Se desarrolló bajo una arquitectura cliente-servidor usando herramientas modernas, lo que permitió crear un sistema ordenado y funcional.

El uso de Next.js facilitó mucho el trabajo, ya que permitió manejar tanto el frontend como el backend en un solo proyecto. Esto hizo más sencilla la integración de todas las partes.

En el frontend, se creó una interfaz tipo chat, fácil de usar y entender. Gracias a React, los mensajes se actualizan en tiempo real sin necesidad de recargar la página, lo que mejora la experiencia del usuario.

Por otro lado, el backend se construyó con API Routes, encargándose de recibir las preguntas del usuario, enviarlas al modelo de inteligencia artificial y devolver respuestas adecuadas. Aquí es donde realmente sucede la “magia” del sistema.

La integración con Ollama fue una parte clave, ya que permite generar respuestas automáticamente, simulando a un tutor. Aunque al inicio hubo algunos retrasos en las respuestas, se fueron corrigiendo con ajustes durante el desarrollo.

Para la base de datos, se utilizó Prisma junto con SQLite, lo cual simplificó mucho el proceso. No fue necesario usar algo complejo, pero sí lo suficiente para guardar las conversaciones correctamente.

En las pruebas, el sistema se comportó de forma estable, sin errores graves. Aun así, se detectaron áreas de mejora, como hacer más rápidas las respuestas y mejorar cómo se maneja el contexto de las conversaciones.

En resumen, técnicamente es un proyecto sólido, bien armado y con potencial para seguir creciendo.

Evaluación financiera

En cuanto al dinero, GarzaMath destaca por ser un proyecto de bajo costo. A diferencia de otros sistemas, aquí no se necesitó hacer grandes inversiones.

No fue necesario pagar licencias ni contratar servicios externos, ya que todo lo que se utilizó es de acceso libre. Esto incluye tanto las herramientas de desarrollo como la base de datos y el modelo de inteligencia artificial.

El único requisito es contar con una computadora que pueda soportar el modelo de IA de manera local. Aunque esto puede ser una limitante en equipos muy básicos, en general no se necesita infraestructura costosa.

Además, al no depender de servicios en la nube, los costos operativos se reducen aún más. Esto lo vuelve muy atractivo para escuelas que quieren implementar tecnología sin gastar demasiado.

Si en el futuro se quisiera hacer más grande el proyecto, sí habría que invertir en servidores más potentes o servicios externos. Pero en su estado actual, es totalmente viable.

Conclusión de la evaluación del proyecto

En conjunto, la evaluación muestra que GarzaMath es un proyecto funcional y viable en varios sentidos. Socialmente, ayuda a mejorar el aprendizaje del álgebra; técnicamente, está bien estructurado; y financieramente, es accesible.

Además, tiene mucho potencial para seguir evolucionando con nuevas funciones y mejoras.

En pocas palabras, GarzaMath no solo cumple con lo que se propuso, sino que también representa una propuesta innovadora que combina tecnología e inteligencia artificial para apoyar a los estudiantes. Demuestra que sí es posible crear herramientas útiles, accesibles y acordes a las necesidades actuales.

Ilustración 22 Logo GarzaMath

Resultados obtenidos en el proyecto

Definición de requerimientos y casos de uso

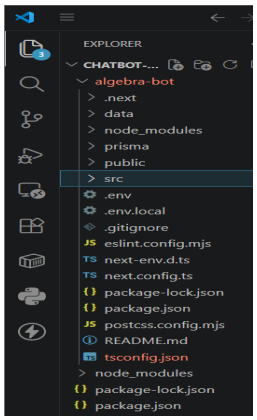
Como primer resultado importante, se logró identificar y organizar las necesidades del Centro de Cómputo Académico de la UAEH. A partir de esto, se generó un catálogo de requisitos tanto funcionales como no funcionales, que definen cómo debe comportarse el chatbot de apoyo académico.

También se establecieron los casos de uso más relevantes, dando prioridad a los que tienen mayor impacto para los estudiantes, como consultas sobre inscripciones, dudas relacionadas con servicios de red y apoyo en laboratorios.

Este análisis fue clave para diseñar el flujo de conversación del chatbot. De hecho, se obtuvo un documento de especificaciones que sirvió como base para todo el desarrollo posterior. Además, al aplicar técnicas de ingeniería de requisitos, se aseguró que el sistema cumpliera con estándares de calidad adecuados.

Diseño de arquitectura y estructura técnica

Otro resultado importante fue el diseño de la arquitectura del sistema, basado en el enfoque de Arquitectura Limpia (Clean Architecture). Esto permitió separar claramente la lógica del negocio de los detalles técnicos de implementación.



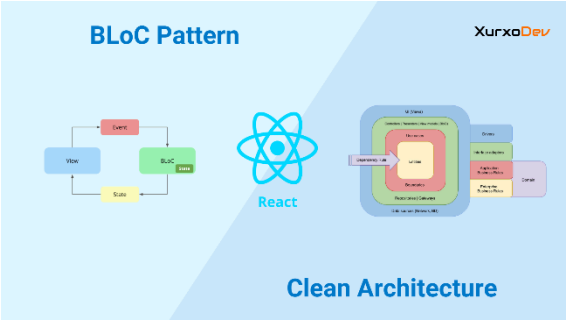


Ilustración 23 Arquitectura Limpia

Gracias a esta estructura, el sistema no depende directamente de tecnologías específicas, lo que facilita su mantenimiento y posibles cambios en el futuro.

La arquitectura se organizó en cuatro capas principales:

- **Entidades:** donde se definen los elementos básicos del sistema.
- **Casos de uso:** que contienen la lógica de interacción académica.
- **Adaptadores:** encargados de conectar el sistema con herramientas como FastAPI y React.
- **Frameworks:** donde se integran tecnologías externas, como la base de datos SQLite.

Desarrollo de la API y lógica del backend (Node.js & TypeScript)

Uno de los resultados más sólidos del proyecto fue la construcción de un backend completamente funcional utilizando Node.js junto con TypeScript. Elegir TypeScript como lenguaje principal ayudó bastante, ya que al trabajar con tipado estático se redujeron errores en tiempo de ejecución y se facilitó la definición clara de las estructuras de datos que se usan para comunicarse con el frontend.

Para la parte de servicios, se implementó una API tipo REST (apoyada en el framework seleccionado), diseñada para ser eficiente y responder rápido. Como resultado, se desarrollaron varios endpoints específicos que reciben las solicitudes de los estudiantes, procesan el lenguaje natural y consultan la base de datos para devolver información académica confiable en cuestión de milisegundos.

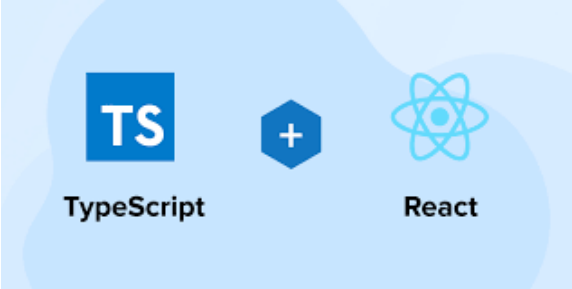


Ilustración 24 Framework utilizado para el desarrollo del proyecto

Otro punto importante fue la integración de la base de datos con SQLite, que permitió resolver varias necesidades clave. Por ejemplo, se pueden guardar las sesiones de los usuarios para tener un historial de consultas, lo que ayuda a mejorar la experiencia. También se optimizaron las respuestas al identificar preguntas frecuentes, reduciendo así la carga del sistema. Además, al tratarse de una base de datos ligera, resulta adecuada para las necesidades del Centro de Cómputo Académico de la UAEH sin requerir infraestructura compleja.

Por último, todo el backend se desarrolló siguiendo un enfoque asíncrono. Esto es fundamental, ya que permite que el chatbot atienda varias solicitudes al mismo tiempo sin afectar su rendimiento. Gracias a esto, el sistema puede mantenerse estable incluso en momentos de alta demanda, como en periodos de inscripciones o exámenes.

Ilustración 24 Referencia a SQLite

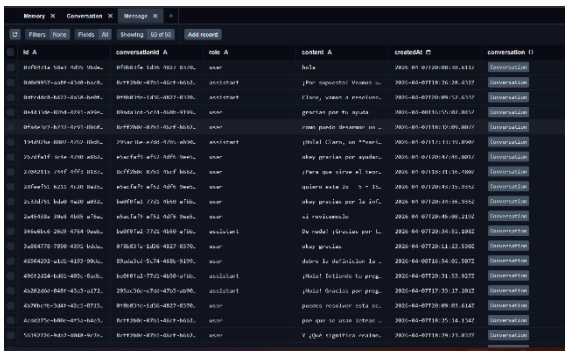


Ilustración 25 Vista general de Prysma (donde se encuentra la base de datos).

.

Interfaz de usuario y experiencia del estudiante (frontend)

En cuanto al frontend, se desarrolló una aplicación web interactiva utilizando React y TypeScript. Se buscó que la interfaz fuera clara, sencilla y fácil de usar, para que los estudiantes puedan comunicarse sin complicaciones con el sistema.

El uso de TypeScript ayudó a tener un código más seguro y estable, evitando errores comunes. Por su parte, la interfaz permite visualizar los mensajes de forma ordenada, incluir contenido multimedia en las respuestas y navegar de manera fluida.

Este punto es especialmente importante, ya que es la parte con la que los usuarios interactúan directamente.

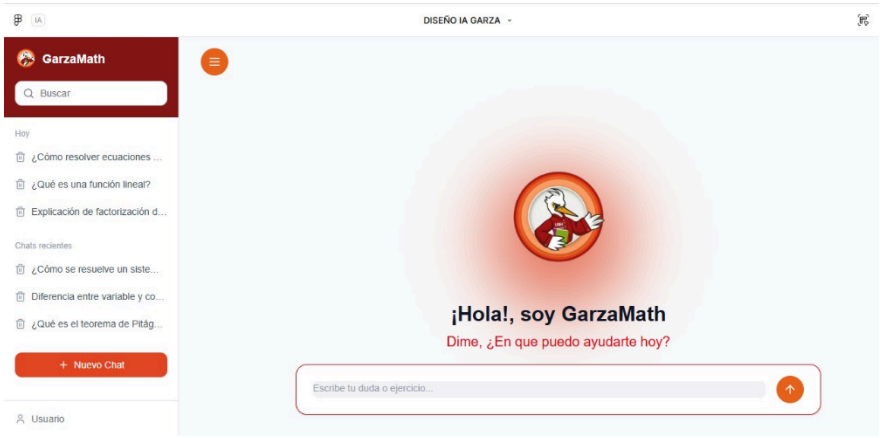


Ilustración 26 Interfaz Final GarzaMath.

Aseguramiento de calidad y validación

Por último, se realizó un proceso de pruebas para asegurar que todo funcionara correctamente. Se utilizó PyTest para generar un reporte detallado, incluyendo pruebas unitarias del backend y pruebas de integración entre el frontend y la API.

Gracias a esto, se pudieron hacer ajustes que mejoraron la estabilidad del sistema, logrando un funcionamiento confiable y con muy pocos errores.

Análisis y discusión de los resultados

Interpretación del alcance y casos de uso

Definir el alcance y los casos de uso del chatbot de apoyo académico no fue solo un trámite, sino una parte clave para entender cómo se estaba comunicando el Centro de Cómputo de la UAEH con los estudiantes. A partir del análisis de los requerimientos, se pudo ver que la mayoría de las dudas que llegan son sobre procesos repetitivos, los cuales realmente no necesitan atención directa de una persona.

En este sentido, el chatbot funciona como un filtro bastante eficiente, ya que atiende este tipo de consultas y libera carga de trabajo al personal administrativo. Esto les permite enfocarse en situaciones más complejas que sí requieren atención personalizada.

Reflexión sobre la arquitectura limpia y el stack tecnológico

El uso de Arquitectura Limpia resultó ser una decisión muy acertada dentro del proyecto. A diferencia de otros enfoques donde todo queda muy ligado entre sí, aquí se logró una estructura más flexible y fácil de adaptar a cambios.

Por ejemplo, si en algún momento se decide cambiar SQLite por una base de datos más robusta como PostgreSQL, no sería necesario rehacer toda la lógica del sistema, ya que esta se mantiene independiente.

Además, la combinación de tecnologías como FastAPI, TypeScript y React funcionó bastante bien en conjunto, logrando un sistema rápido y confiable. En particular, TypeScript ayudó a detectar errores desde etapas tempranas, lo que redujo considerablemente el tiempo de corrección durante el desarrollo. Esto es especialmente importante en un sistema que debe estar disponible todo el tiempo.

Discusión sobre la interfaz y experiencia de usuario

En cuanto al frontend, los resultados muestran que la facilidad de uso es clave para que una herramienta tecnológica realmente sea adoptada. Se observó que una interfaz sencilla y directa ayuda a que los estudiantes encuentren lo que necesitan sin complicarse.

Al reducir la cantidad de pasos o menús innecesarios, también se disminuye la carga mental del usuario, lo que mejora la experiencia general. Más allá de ser solo un software, el chatbot se convierte en un nuevo canal de comunicación entre la universidad y los estudiantes, aportando también a una imagen más moderna de la institución.

Sugerencias y ampliación de la práctica profesional

A partir de los resultados obtenidos, también surgen algunas ideas para mejorar y ampliar el alcance del proyecto:

- **Integración de inteligencia artificial generativa:** Aunque el sistema actual funciona bien con respuestas estructuradas, se podría mejorar incorporando modelos de lenguaje que permitan interacciones más naturales y cercanas con los estudiantes.
- **Expansión a dispositivos móviles:** Sería buena idea llevar este sistema a una app móvil o integrarlo con plataformas como WhatsApp o Telegram, para que más estudiantes puedan acceder fácilmente.
- **Análisis de datos:** Implementar herramientas de análisis ayudaría a identificar cuáles son las dudas más comunes en diferentes momentos del año, como inscripciones o exámenes. Esto permitiría anticiparse a las necesidades de los estudiantes y mejorar la comunicación institucional.

Recomendaciones y sugerencias

A partir de los resultados obtenidos durante el desarrollo de GarzaMath, se identificaron diversas áreas de oportunidad que, en futuras etapas, podrían contribuir significativamente a mejorar la funcionalidad, eficiencia y pertinencia del sistema como herramienta de apoyo académico.

En primer lugar, se detecta la necesidad de optimizar el manejo del contexto en las conversaciones. Si bien el chatbot ofrece respuestas adecuadas ante consultas individuales, presenta limitaciones al mantener la coherencia en interacciones prolongadas. En este sentido, se recomienda implementar un mecanismo que permita el envío y procesamiento del historial completo de la conversación por parte del modelo de inteligencia artificial, con el propósito de generar respuestas más consistentes, contextualizadas y alineadas con el flujo conversacional.

Otro aspecto relevante corresponde al tiempo de respuesta del sistema. A pesar de las optimizaciones realizadas, aún se observan retrasos en la generación de respuestas en determinados escenarios. Para atender esta problemática, se sugiere el uso de modelos de menor carga computacional, la optimización de la configuración del entorno de ejecución (como Ollama) y la mejora en la gestión de solicitudes dentro del backend.

De igual manera, resulta fundamental fortalecer la interfaz de usuario, orientándola hacia una experiencia más intuitiva y accesible. Entre las mejoras propuestas se encuentran la incorporación de indicadores de carga durante el procesamiento de respuestas, así como la optimización del diseño visual de los mensajes, con el fin de facilitar su lectura y comprensión por parte del usuario.

En relación con las funcionalidades del sistema, se considera pertinente ampliar las capacidades del chatbot. Además de la resolución de consultas, se propone integrar módulos que permitan la generación de ejercicios, la presentación de ejemplos prácticos y la evaluación de respuestas del usuario. Estas funcionalidades contribuirían a consolidar a GarzaMath como una herramienta integral de aprendizaje autónomo.

Asimismo, se recomienda la implementación de un sistema de gestión de usuarios. Esta funcionalidad permitiría almacenar de manera individual el historial de interacción de cada estudiante, facilitando el seguimiento de su progreso académico y habilitando la personalización de la experiencia de aprendizaje.

Además de las recomendaciones previamente expuestas, se plantean los siguientes puntos de mejora:

- Integración de modelos avanzados de IA: Se recomienda continuar con la integración de modelos de inteligencia artificial más robustos para mejorar la precisión y calidad de las respuestas académicas.
- Optimización de la experiencia de usuario (UX): Se sugiere realizar mejoras continuas en la interfaz para asegurar que la interacción entre el estudiante y el chatbot sea lo más sencilla e intuitiva posible.
- Implementación de un sistema de logs: Se recomienda desarrollar un sistema de registro de eventos (logs) que permita monitorear el comportamiento del sistema y facilitar la detección de errores en tiempo real.
- Escalabilidad del proyecto: El sistema debe diseñarse considerando su crecimiento, permitiendo la incorporación de nuevos temas matemáticos más allá del álgebra básica.
- Reducción de tiempos de respuesta: Es fundamental continuar optimizando el backend para minimizar problemas de latencia o interrupciones durante la interacción.

- Evaluación continua: Se sugiere mantener un proceso sistemático de pruebas con usuarios reales (estudiantes de nivel secundaria y medio superior) para validar de manera constante la efectividad del sistema como herramienta de apoyo académico.

Finalmente, se propone considerar la migración del sistema a un entorno en la nube, lo cual permitiría ampliar su accesibilidad, eliminando la dependencia de instalaciones locales y favoreciendo su escalabilidad y disponibilidad para un mayor número de usuarios.

En conclusión, si bien GarzaMath cumple con los objetivos planteados en su desarrollo inicial, la implementación de las mejoras aquí descritas permitiría fortalecer sus capacidades, incrementar su eficiencia y consolidarlo como una herramienta con mayor impacto en el ámbito educativo.

Conclusiones

A lo largo del desarrollo de GarzaMath, se abordó una problemática ampliamente identificada en el ámbito educativo: la dificultad que presentan numerosos estudiantes de nivel secundaria y medio superior en el aprendizaje del álgebra, particularmente cuando no cuentan con apoyo fuera del entorno escolar. Esta situación impacta de manera directa en su nivel de comprensión y, en consecuencia, en su desempeño académico.

A partir de este contexto, se estableció como objetivo principal el desarrollo de un chatbot académico basado en inteligencia artificial, orientado a la resolución de dudas en álgebra. Para su cumplimiento, se plantearon diversos objetivos específicos, entre los que destacan el diseño de la arquitectura del sistema, el desarrollo de la interfaz de usuario, la implementación del backend, la integración del modelo de inteligencia artificial y la gestión del almacenamiento de las conversaciones.

Asimismo, se formuló la hipótesis de que una herramienta de este tipo podría contribuir a mejorar el aprendizaje al ofrecer un recurso accesible, interactivo y con capacidad de respuesta inmediata. Con base en las pruebas realizadas, se puede afirmar que dicha hipótesis se cumple en gran medida, ya que el sistema permite resolver dudas de forma ágil y proporciona explicaciones claras que favorecen la comprensión de los estudiantes.

En cuanto a los resultados obtenidos, se logró desarrollar una aplicación web funcional que integra tecnologías actuales tanto del desarrollo web como de la inteligencia artificial. El sistema permite la interacción mediante un chat, la generación automática de respuestas y el almacenamiento de conversaciones, cumpliendo con los requerimientos establecidos desde las etapas iniciales del proyecto. Además, durante las pruebas realizadas, se observó un desempeño adecuado del chatbot, evidenciando su utilidad en la resolución de ejercicios de álgebra.

Desde la perspectiva tecnológica, se concluye que el uso de inteligencia artificial local, mediante herramientas como Ollama, representa una alternativa viable y efectiva para fortalecer el aprendizaje autónomo. Esta implementación no solo garantiza independencia de servicios externos, sino que también permite un mayor control sobre el procesamiento y la gestión de la información.

En relación con el proceso de desarrollo, la adopción de metodologías ágiles, particularmente el enfoque basado en Scrum, resultó fundamental para asegurar un desarrollo flexible y eficiente. Esta metodología permitió la identificación y corrección temprana de errores técnicos, así como una mejor organización de las actividades durante el periodo de estadía.

Por otro lado, es importante destacar que la herramienta desarrollada no tiene como finalidad sustituir la labor docente, sino complementar. GarzaMath funciona como un apoyo adicional que permite a los estudiantes reforzar su aprendizaje de manera autónoma, facilitando que lleguen al aula con una mejor comprensión de los temas al haber resuelto previamente sus dudas.

Respecto al cumplimiento de los objetivos, el proyecto puede considerarse satisfactorio, ya que se logró implementar una solución funcional que responde a la problemática identificada. No obstante, también se detectaron áreas de mejora, tales como la optimización en los tiempos de respuesta y el fortalecimiento del manejo del contexto en las conversaciones, lo que abre la posibilidad de futuras mejoras y líneas de desarrollo.

En conclusión, GarzaMath constituye una propuesta innovadora que demuestra el potencial de la inteligencia artificial aplicada al ámbito educativo. El sistema no solo cumple con su objetivo principal, sino que además establece una base sólida para su evolución, permitiendo el desarrollo de futuras mejoras que incrementen su impacto en el aprendizaje autónomo mediante el uso de herramientas digitales.

Referencias

- *Holmes, W., Bialik, M., & Fadel, C. (2019). Inteligencia artificial en la educación: Promesas e implicaciones para la enseñanza y el aprendizaje. Center for Curriculum Redesign. https://curriculumredesign.org/wp-content/uploads/AIED-Report-2019.pdf*
- *Jurafsky, D., & Martin, J. H. (2020). Procesamiento del lenguaje natural y del habla (3ª ed. preliminar). https://web.stanford.edu/~jurafsky/slp3/*
- *Kieran, C. (2007). La enseñanza y el aprendizaje del álgebra. En F. Lester (Ed.), Segundo manual de investigación en educación matemática (pp. 707–762). Springer. https://link.springer.com/chapter/10.1007/978-1-59385-260-9_11*
- *Meta. (2024). Llama 3 model overview. https://ai.meta.com*
- *Mozilla Developer Network. (2024). JavaScript guide. https://developer.mozilla.org*
- *Ollama. (2024). Ollama documentation. https://ollama.com*
- *OpenAI. (2023). Introducción a los modelos de lenguaje y chatbots. https://openai.com*
- *Pressman, R. S. (2014). Ingeniería de software: Un enfoque práctico (7ª ed.). McGraw-Hill.*
- *Prisma. (2024). Documentación oficial de Prisma ORM. https://www.prisma.io/docs*
- *Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.*
- *Sommerville, I. (2016). Ingeniería de software (10ª ed.). Pearson.*
- *SQLite. (2024). SQLite documentation. https://www.sqlite.org/docs.html*
- *Vercel. (2024). Next.js documentation. https://nextjs.org/docs*
- *Winkler, R., & Söllner, M. (2018). Liberando el potencial de los chatbots en la educación. https://www.researchgate.net/publication/322817270*

- *Zimmerman, B. J. (2002). Convertirse en un aprendiz autorregulado: Una visión general. Theory Into Practice, 41(2), 64–70.* https://www.tandfonline.com/doi/abs/10.1207/S15430421TIP4102_2

Anexos

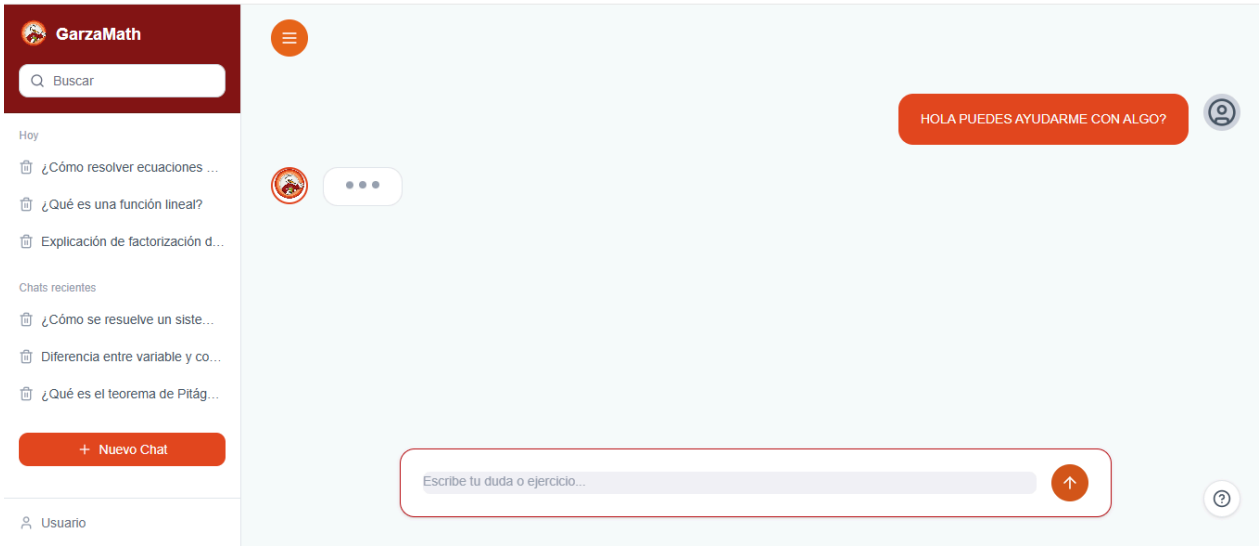
Anexo 1

Manual de usuario

1. Se tiene la vista general del proyecto, con el que al escribir una petición en el recuadro con borde naranja nos responderá de acuerdo a como es el tutor.

2. Al escribir la petición, basta con dar “intro” o click en el icono naranja de lado derecho de la petición.

3. Al hacer lo mencionado anteriormente, se mostrará una animación donde el modelo está “pensando” como se muestra en la imagen.



4. Al terminar de pensar, nos arroja la respuesta acorde a lo que hayamos solicitado al modelo.

5. Finalmente podemos dejar el chat guardado para futuras conversaciones ya que éste modelo ya cuenta con memoria, y si se desea, se puede eliminar el chat que no se ocupe presionando el icono de “basura” que está del lado izquierdo de cada título de chat.
6. También, se puede crear chat nuevo presionando el botón naranja de “nuevo chat” ubicado en la parte inferior, y asi crear un nuevo chat y volver a realizar peticiones.

Anexo 2

Manual de instalación de GarzaMath.

Requisitos del sistema.

Para el correcto funcionamiento del sistema el equipo debe contar con los siguientes componentes previamente instalados:

- npm (incluido con node)
- Node.js versión 18 o superior
- Git
- Ollama (para ejecución de modelos de IA)
- Sistema Operativo Windows

Instalación del sistema

1. Clonación del repositorio: el primer paso consiste en obtener el código fuente del repositorio, para ello ejecutaremos el siguiente comando en la terminal:

```
git clone <https://github.com/ivanperezt270-wq/GarzaMath.git>

cd chatbot-algebra-clean/algebra-bot
```

2. Dentro de la carpeta del proyecto ejecutaremos el siguiente comando:

```
npm install
```

Este comando instalará las dependencias necesarias incluyendo next.js , prisma, librerías de autenticación, cliente de base de datos.

3. El sistema utiliza Prisma con SQLite, por lo que se requiere inicializar la base de datos con el siguiente comando:

```
npx prisma generate
```

4. Ahora ejecutaremos las migraciones con el comando:

npx prisma migrate dev

Esto creará automáticamente la base de datos (dev.db) ubicada en: /prisma/dev.db

5. Se debe crear un archivo .env en la raíz del proyecto con una configuración similar a:

DATABASE_URL="file:./dev.db"

OLLAMA_MODEL=gemma:2b

Estas variables permiten conectar la base de datos y definir el modelo de lenguaje utilizado.

6. Configuración de Ollama

6.1 Ollama debe estar instalado en el sistema.

6.2 Descargar el modelo ejecutando el siguiente comando:

ollama pull gemma:2b

6.3 Verificar las versiones instaladas con el comando

ollama list

7. Ahora se realizará la ejecución, el sistema se ejecutará como una aplicación next.js con el siguiente comando.

npm run dev

Este comando ayudará a levantar el servidor en *http://localhost:3000*

8. Ahora verificaremos el funcionamiento para ello debemos:

- Acceder a: *http://localhost:3000*

- Realizar los pasos mencionados en el Anexo 1 “Manual de Usuario” pag. 62 - 64
- Verificar que los mensajes se guardan correctamente, o bien, probar con el endpoint /api/db/ping para validar la conexión con la base de datos.

Problemas comunes que pueden surgir durante la instalación:

1. Error al conectar con la base de datos

- Verificar que Prisma esté correctamente configurado
- Ejecutar nuevamente migraciones

2. Ollama no responde

- Asegurarse de que el modelo esté descargado
- Verificar que Ollama esté en ejecución

3. Error al Ejecutar dependencias
    - Al ejecutar npm install, algunas dependencias no se instalan correctamente.
    - Ejecutar *npm cache clean --force* y después *npm install*

4. Error al generar cliente prisma
    - Archivo schema.prisma mal configurado
    - Dependencias incompletas

    Solución:

        - ejecutar *npx prisma generate*

5. Error en migraciones de bases de datos.
    - Ruta incorrecta en DATABASE_URL
    - Conflictos con migraciones previas

    Solución:

        - Ejecutar npx prisma migrate reset , este comando elimina y recrea la base de datos.
6. Puerto en uso.
    - El sistema no inicia porque el puerto 3000 ya está ocupado.

    Solución:

        - ejecutar en otro puerto con el código npm run dev -- -p 3001
7. Error de conexión con Ollama que hace que el chatbot no responda.
    - Ollama no está en ejecución
    - Modelo no descargado
    - Nombre incorrecto del modelo en .env

    Solución:

        - ejecutar ollama list y ollama run gemma:2b
8. El sistema no detecta valores definidos en .env.
    - Archivo mal ubicado
    - Cambios sin reiniciar el servidor

    Solución:

        - Verificar que .env esté en la raíz del proyecto
        - Reiniciar el servidor con npm run dev

9. Errores inesperados al ejecutar el sistema.
    - Cambios en versiones de librerías

      Solución:

        - Eliminar e instalar nuevamente con los siguientes comandos rm -rf node_modules package-lock.json y npm install
10. El chatbot tarda en responder.
    - Modelo de Ollama pesado
    - Recursos limitados del equipo

      Solución

        - Usar un modelo más ligero como Gemma 2b con el siguiente comando ollama pull gemma:2b
11. Error al acceder desde el navegador (la página no carga)
    - Servidor no iniciado
    - Firewall o bloqueo del puerto

      Solución:

        - Verificar ejecución del servidor
        - Revisar configuración de red

La identificación y documentación de estos posibles errores permite mejorar la robustez del sistema, facilitando su implementación en distintos entornos y reduciendo tiempos de configuración.